COAL Lab – 3

Task 1:

Predicted Values:

mov eax, sensorA ; EAX = 120

add eax, sensorB ; EAX = 165

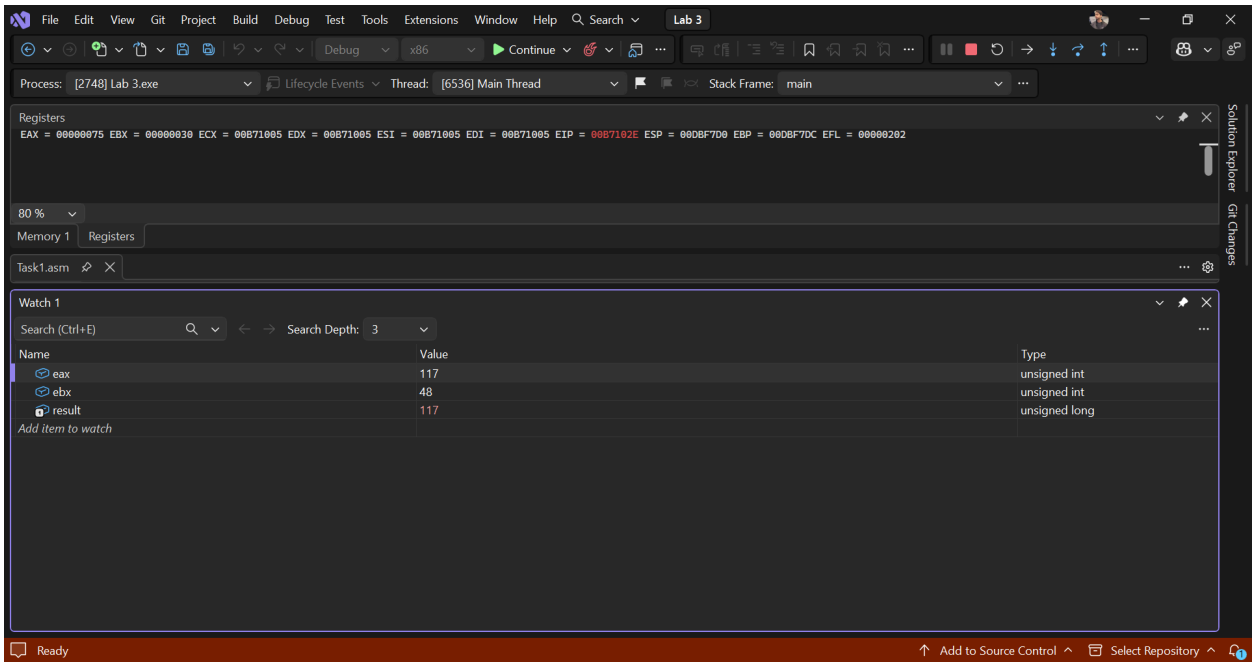
mov ebx, sensorC ; EBX = 30

add ebx, sensorD ; EAX = 48

sub eax, ebx ; EAX = 117

mov result, eax ; result = 117

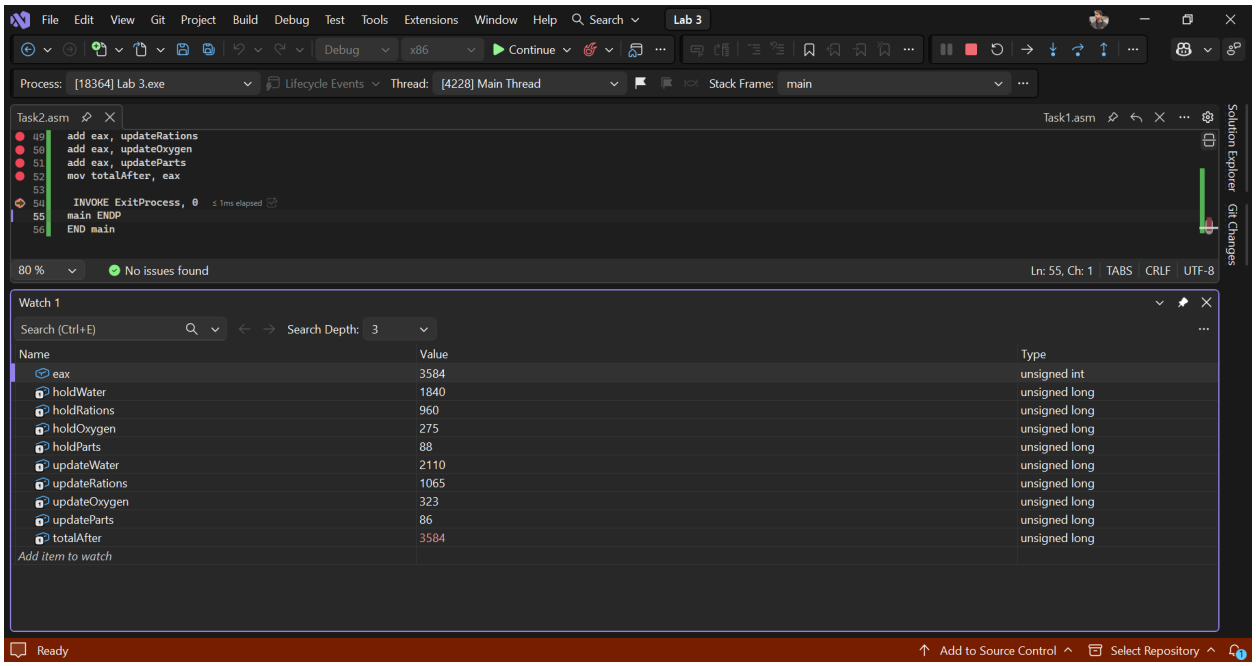
Screenshot:



Task 2:

Variable	Predicted	Observed
holdWater (original)	1840	1840
holdRations (original)	960	960
holdOxygen (original)	275	275
holdParts (original)	88	88
updated water	2110	2110
updated rations	1065	1065
updated oxygen	323	323
updated parts	86	86
totalAfter	*3584*	*3584*

Screenshot:



Task 3:

Object	TYPE	LENGTHOF	SIZEOF
frameID	4	1	4
sensorLog	2	64	128
statusBits	1	16	16
stationName	1	13	13
spare	4	24	96
		Total bytes:	**257**

3. Add all SizeOF

The observed values were the same as mentioned in the above table.

Adding all sizeof resultantly reveals **257 bytes** of memory which is occupied in the whole process. Since we've 512 bytes of memory available, this model will fit in easily.

4. Quick Answers

- **frameID** is a single DWORD variable with only one value, hence it will only take the size of one element which is **4 bytes.**
- **sensorLog** holds 64 readings but occupies more than 64 bytes but it occupies 128 bytes because it's a DW/WORD type which occupies 2 byte of space per element. Since it's a whole array of 64 elements, it occupies 64*2 = **128 bytes**
- Removing **0** from end of stationName will remove the null terminator from the end of the string, and since it was an array of BYTE type characters, it will reduce size by **1 Byte**. The type will remain the same but the length and size will reduce by 1 because null terminator is also an element taking up exactly 1 byte memory for being a BYTE type.

Task 4:

#	Error code (or “none”)	Which rule is broken	Corrected line

1	mov total, countA	Memory to Memory not allowed	mov eax, tota
			add eax, 25
			mov total, eax

2	mov **eax**, limitByte	Register and variable should be same size	limitByte DD 200 ; or DWORD instead of BYTE
3	add 20, **al**	Wrong instruction	add al, 20
4	mov **ebx**, SIZEOF slots mov slotCount, **ebx**	Expects number of elements in slots, we get total size instead	mov ebx, LENGTHOF slots

Screenshot: